TripPlanner software docs 1

TripPlanner System Documentation

1. Introduction

1.1 Purpose of the Document

This document provides a comprehensive technical specification of the TripPlanner system, derived exclusively from inspection of the source code repository.

1.2 Scope of the Document

1.3 System Overview

TripPlanner is a collaborative trip planning application designed to help groups organize shared travel itineraries.

Core Functionalities:

- User authentication and account management
- Trip creation and collaborative editing
- Real-time chat (REST polling) and Meeting capabilities
- Proposal-based event scheduling system
- Voting mechanism for proposals with automatic merging capabilities
- Geo-distributed support (Cold and Warm DC)
- Multi-tenancy (partitions keyed with PLESETTER, DATABASE, MEMBERCRC)

Application Type: Stateless application with 12-factor-compliant architecture, deployed as Docker containers on Kubernetes.

1.4 Hardware and Software Requirements

Software Requirements:

Component	Specification
Programming Language	Java 21
Build Tool	Gradle 8.14.3 Maven
JP Framework	Jooq0.21.20
Security	Spring Security Crypto 6.5.2
Database Driver	PostgreSQL 42.7.7
Database Pool	HikariCP (JDBC Driver 6.4.0)
Graph Visualization	GraphStream 2.2.0
Database	PostgreSQL

Hardware Requirements

Platform-independent (standard Java application requirements apply)

1.5 Related or Comparable Systems

System	Similarity	Differences
Trip2	Trip planning, collaboration, voting, chat	Trip2 focuses on individual trips, TripPlanner multi-tenant group collaboration
Google Trips	POI and itinerary management	TripPlanner implements graph-based itineraries with proposal voting

2. User Stories

Terms and additional definitions at the bottom are marked with asterisks (*)

US1: Route Creation

As a logged in user
I want to create a route
So that I can organize a collaborative trip

US2: Search Points of Interest

As a logged in user
I want to find POIs by name, tags, and location
So that I can add them to my trip route

US3: Propose Event Addition

As a crew member
I want to propose adding an event to a branch
So that my group can consider including it in the trip

US4: Like/Unlike Proposals

As a crew member
I want to express my preference for proposals created by other members
So that our voting algorithm can prioritize popular options

US5: Cast Votes

As a crew member
I want to express my preference for the graph
So that our vote count computes multi-path Dijkstra

3. Dictionary

Term	Definition
Route	Logical group with one administrator and many members; each has a dedicated chat room
POI	Point of interest - relevant attracting points during a trip (monuments, restaurants, etc.)
Rating	Popularity indicator from 1-5 stars
Category	Hashtags like #city #LIKE #BEACHWALKS, FOOD, GASTRONOMY, #museum
Branch	Unordered sequence of events in chronological order
Proposal	Facility for a branch to be created, to be changed, accepted by administrators
Event	Trip stage where members plan to be at a POI between defined start/end times

4. Use Cases

5. Internal Steps: UC10 - Propose Add Event

Primary Actor

Route Member

Preconditions

- User is logged in
- User is a route member

Main Flow

1. User selects "propose new event" option
2. User selects target branch
3. System displays POI filter interface
4. User applies filters (optional)
5. System filters POIs based on criteria and room destination
6. System displays filtered POI list
7. User selects a POI from results
8. User enters start time
9. User enters end time
10. System validates time format
11. System validates time logic
12. System creates proposal with selected branch, POI, and interval
13. System persists proposal
14. System notifies other branch members

Post-conditions

The system must track who created each proposal and prevent duplicate likes.

FR6: Proposal Voting

The system must track which users have liked each proposal and prevent duplicate likes.

FR7: User Authentication

The system must hash all user passwords using BCrypt algorithm before storage.

FR2: Route Code Generation

The system must generate unique human-readable route codes.

FR3: Event Time Overlap Prevention

The system must prevent overlapping events within the same branch.

FR4: Graph Cycle Prevention

The system must prevent cycles in event dependencies.

FR8: Chronological Branch Connection

The system must ensure events are connected in time by the graph.
So that we can resolve different plan options

US6: Connect Branches

As a crew administrator
I want to connect two "endpoints" in the graph
So that we can build complex multi-path itineraries

6. Internal Steps: UC11 - Propose Add Event

Primary Actor

Route Member

Preconditions

- User is logged in
- User is a route member

Main Flow

1. User selects "propose new event" option
2. User selects target branch
3. System displays POI filter interface
4. User applies filters (optional)
5. System filters POIs based on criteria and room destination
6. System displays filtered POI list
7. User selects a POI from results
8. User enters start time
9. User enters end time
10. System validates time format
11. System validates time logic
12. System creates proposal with selected branch, POI, and interval
13. System persists proposal
14. System notifies other branch members

Post-conditions

The system must track who created each proposal and prevent duplicate likes.

FR6: Proposal Voting

The system must track which users have liked each proposal and prevent duplicate likes.

7. Use Cases

8. Internal Steps: UC10 - Propose Add Event

Primary Actor

Route Member

Preconditions

- User is logged in
- User is a route member

Main Flow

1. User selects "propose new event" option
2. User selects target branch
3. System displays POI filter interface
4. User applies filters (optional)
5. System filters POIs based on criteria and room destination
6. System displays filtered POI list
7. User selects a POI from results
8. User enters start time
9. User enters end time
10. System validates time format
11. System validates time logic
12. System creates proposal with selected branch, POI, and interval
13. System persists proposal
14. System notifies other branch members

Post-conditions

The system must track who created each proposal and prevent duplicate likes.

FR6: Proposal Voting

The system must track which users have liked each proposal and prevent duplicate likes.

9. Storyboards

Room Management Interface

Trip Management Interface

Create Event Proposal Interface

7. Class Diagram

8. Design Patterns

Singleton Pattern

Decorator Pattern

Client
Base Object
Decorator A
Decorator B
Decorator C
Add Logging
Add Caching
Add Metrics
Add Rate Limiting
Add Authentication
Add Compression
Add Encryption
Add Retry Logic
Add Circuit Breaker
Add Tracing
Add Validation
Add Transaction Management
Add Connection Pooling
Add Load Balancing
Add Failover